

Gaussian Mixture Models

Replace hard spherical clusters with soft probabilistic components

Unsupervised Learning · Foundations of Machine Learning



A Mixture Is a Generative Story

Choose a component — Sample k according to its mixing weight π_k .

Generate a point — Draw x from that component's Gaussian.

Flexible density — Several simple Gaussians form a multimodal distribution.

$$p(x) = \sum_{k=1}^K \pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k), \quad \sum_k \pi_k = 1$$

Responsibilities Are Soft Assignments

Posterior membership — Each point receives a probability for every component.

Bayes' theorem — Prior × likelihood, normalized across components.

Boundary uncertainty — Ambiguous points can split responsibility between clusters.

$$\gamma_{ik} = P(z_i = k \mid x_i) = \frac{\pi_k \mathcal{N}(x_i \mid \mu_k, \Sigma_k)}{\sum_j \pi_j \mathcal{N}(x_i \mid \mu_j, \Sigma_j)}$$

Expectation–Maximization

E-step — Compute responsibilities using current parameters.

M-step — Update means, covariances, and mixing weights using weighted statistics.

Repeat — Likelihood improves until convergence to a local optimum.

1

Initialize

2

E: soft assign

3

M: update

4

Converged?

$$\mu_k = \frac{\sum_i \gamma_{ik} x_i}{\sum_i \gamma_{ik}}, \quad \pi_k = \frac{1}{n} \sum_i \gamma_{ik}$$

A Worked Responsibility Calculation

Two 1D components: $\pi_1 = \pi_2 = 0.5$, $\mathcal{N}(\mu_1=0, \sigma_1=1)$, $\mathcal{N}(\mu_2=4, \sigma_2=1)$. Point $x=1.5$.

- $\mathcal{N}(1.5 \mid 0, 1) = \frac{1}{\sqrt{2\pi}} e^{-1.5^2/2} \approx 0.1295$
- $\mathcal{N}(1.5 \mid 4, 1) = \frac{1}{\sqrt{2\pi}} e^{-2.5^2/2} \approx 0.0175$
- unnormalized: $.5(.1295) = .0648$ and $.5(.0175) = .0088$
- $\gamma_{i1} = \frac{.0648}{.0648 + .0088} \approx 0.880, \quad \gamma_{i2} \approx 0.120$

```
import numpy as np
from scipy.stats import norm

x = 1.5
pi = [0.5, 0.5]
lik = [norm.pdf(x, 0, 1), norm.pdf(x, 4, 1)]
unnorm = [p * l for p, l in zip(pi, lik)]
resp = [u / sum(unnorm) for u in unnorm]
assert round(resp[0], 2) == 0.88
```

Fit and Inspect a GMM

`predict_proba`

Returns responsibilities whose rows sum to one.

`predict`

Uses the largest responsibility when a hard label is needed.

```
from sklearn.mixture import GaussianMixture

gmm = GaussianMixture(
    n_components=3, covariance_type="full", random_state=0,
)
gmm.fit(X)
responsibility = gmm.predict_proba(X)

assert responsibility.shape == (len(X), 3)
assert np.allclose(responsibility.sum(axis=1), 1)
```

What GMM Fixes—and Costs

More flexible

Full covariance supports tilted, elongated, differently sized clusters and soft membership.

More parameters

Each component estimates a covariance matrix as well as a mean.

Tradeoff

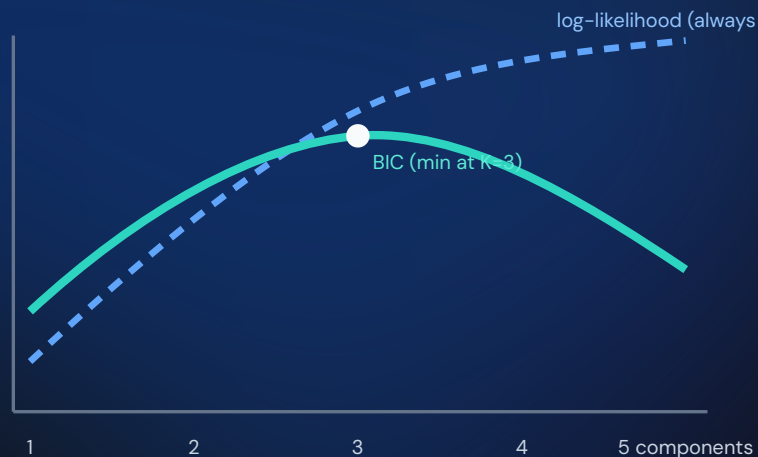
Needs more data and can face numerical instability in high dimension.

Choose Components With Information Criteria

Likelihood rewards fit — More components can always explain the sample better.

AIC / BIC penalize complexity — Extra free parameters must earn their keep.

Decision — Prefer the K with the smallest criterion.



$$\text{BIC} = -2 \log \hat{L} + p \log n, \quad \text{AIC} = -2 \log \hat{L} + 2p$$

Probabilistic Clustering

Mixture

Weighted Gaussian components.

EM

Soft assign, then update.

BIC / AIC

Balance fit and complexity.

Next: neural networks return to supervised learning with composable nonlinear models.